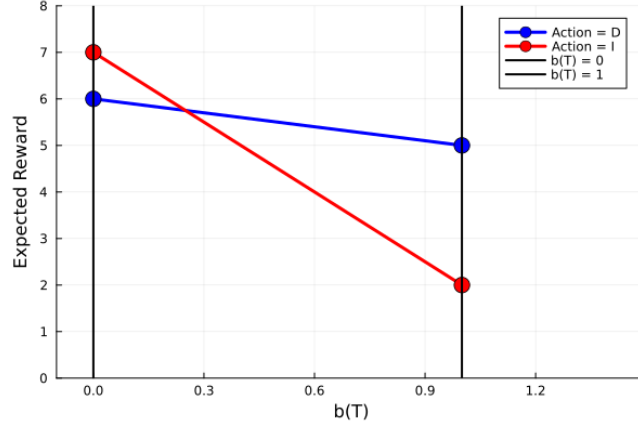


ASEN 5264 - DMU

Homework #5

Bijan Jourabchi

Problem 1:

Figure 1: One Step α -vector

a)

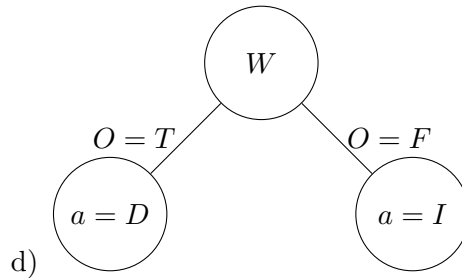
b) With $b(T) = 0.6$, the optimal one step plan is to go with a domestic supplier.

$$\vec{R}^D = \begin{bmatrix} 6 \\ 5 \end{bmatrix} \quad \vec{R}^I = \begin{bmatrix} 9 \\ 2 \end{bmatrix} \quad \vec{b} = \begin{bmatrix} 0.4 \\ 0.6 \end{bmatrix}$$

$$\vec{R}^D \cdot \vec{b} = 5.4 \quad \vec{R}^I \cdot \vec{b} = 4.8$$

Since $\vec{R}^D \cdot \vec{b} > \vec{R}^I \cdot \vec{b}$, the action $a = D$ is the optimal action in the one step lookahead.

c) As computed in part b, the expected reward given her belief is 5.4.



e) Before we can plot the two step α -vector, we must find the two step utility given by the following expression

$$\mathcal{U}^\pi(s) = R(s, \pi) + \sum_{s'} T(s' | s, \pi) \sum_o \mathcal{O}(o | \pi, s') \mathcal{U}^{\pi(o)}(s')$$

Where $\mathcal{U}^{\pi(o)}(s')$ is the one step alpha vector for action $\pi(o)$ is state s' . Evaluating, we get

$$\begin{aligned}\mathcal{U}^{\pi}(F) &= -1 + [T(F | F, \pi)(\mathcal{O}(T | \pi, F)\mathcal{U}^{\pi(T)}(F) + \mathcal{O}(F | \pi, F)\mathcal{U}^{\pi(F)}(F)) + \dots \\ &\quad T(T | F, \pi)(\mathcal{O}(T | \pi, T)\mathcal{U}^{\pi(T)}(T) + \mathcal{O}(F | \pi, T)\mathcal{U}^{\pi(F)}(T))] \\ &= -1 + \mathcal{U}^{\pi(F)}(F) = -1 + 9 = 8 \\ \mathcal{U}^{\pi}(T) &= -1 + [T(F | T, \pi)(\mathcal{O}(T | \pi, F)\mathcal{U}^{\pi(T)}(F) + \mathcal{O}(F | \pi, F)\mathcal{U}^{\pi(F)}(F)) + \dots \\ &\quad T(T | T, \pi)(\mathcal{O}(T | \pi, T)\mathcal{U}^{\pi(T)}(T) + \mathcal{O}(F | \pi, T)\mathcal{U}^{\pi(F)}(T))] \\ &= -1 + \mathcal{U}^{\pi(T)}(T) = -1 + 5 = 4\end{aligned}$$

Thus, the two step α -vector is

$$\vec{\alpha} = \begin{bmatrix} 8 \\ 4 \end{bmatrix}$$

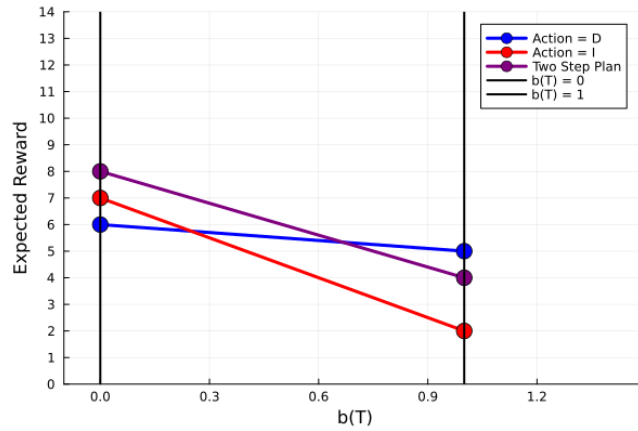


Figure 2: Two Step α -vector

- f) From our plot, it is evident that the two step plan is optimal plan. To find the expected reward, we will follow the same process as part b.

$$\begin{aligned}\vec{b} &= \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} & \vec{R}^D &= \begin{bmatrix} 6 \\ 5 \end{bmatrix} & \vec{R}^I &= \begin{bmatrix} 9 \\ 2 \end{bmatrix} & \vec{R}^W &= \begin{bmatrix} 8 \\ 4 \end{bmatrix} \\ \vec{R}^D \cdot \vec{b} &= 5.5 & \vec{R}^I \cdot \vec{b} &= 5.5 & \vec{R}^W \cdot \vec{b} &= 6\end{aligned}$$

So, the two step plan has an expected reward of 6.

- g) If Jill had a preference to support domestic suppliers, the problem could change considerably. For one, the reward for choosing $a = D$ could have larger rewards in either state. If the rewards for $a = D$ were large enough, it wouldn't even matter what the state is, choosing the domestic supplier will always be better. In this case, the one step plan for $a = D$ would be the optimal policy for any belief.

Problem 2:

With 10,000 MC trials, I get an average return of 38.72 for this POMDP. This can be interpreted as it taking on average 38 steps in the environment until the patient dies, since choosing the wait action always results in a reward of 1.

```

1 using QuickPOMDPs: QuickPOMDP
2 using POMDPTools: Deterministic, Uniform, SparseCat, FunctionPolicy,
   RolloutSimulator, stepthrough, RandomPolicy
3 using Statistics: mean
4 using POMDPs: simulate
5 import POMDPs
6
7 cancer = QuickPOMDP(
8     states = ["Healthy","ISC", "IC", "Death"],
9     actions = ["wait","test","treat"],
10    observations = ["positive","negative"],
11    initialstate = Deterministic("Healthy"),
12    discount = 0.99,
13
14    transition = function(s,a)
15        if s == "Healthy"
16            return SparseCat(["Healthy", "ISC"], [1-0.02, 0.02])
17        elseif a == "treat" && s == "ISC"
18            return SparseCat(["Healthy", "ISC"], [0.6,0.4])
19        elseif a != "treat" && s == "ISC"
20            return SparseCat(["IC","ISC"], [0.1,0.9])
21        elseif a == "treat" && s == "IC"
22            return SparseCat(["IC","Healthy","Death"], [0.6,0.2,0.2])
23        elseif a != "treat" && s == "IC"
24            return SparseCat(["IC","Death"], [0.4,0.6])
25        elseif s == "Death"
26            return Deterministic("Death")
27        end
28    end,
29
30    observation = function(s,a,sp)
31        if a == "test"
32            if sp == "Healthy"
33                return SparseCat(["positive","negative"], [0.05,0.95])
34            elseif sp == "ISC"
35                return SparseCat(["positive","negative"], [0.8,0.2])
36            elseif sp == "IC"
37                return SparseCat(["positive","negative"], [1.0,0])
38            end
39        end
40        if a == "treat"
41            if sp == "ISC" || sp == "IC"
42                return SparseCat(["positive","negative"], [1.0,0])
43            end
44        end
45
46        return SparseCat(["positive","negative"], [0,1.0])
47    end,
48
49    reward = function(s,a)
50        if s == "Death"
51            return 0
52        end

```

```

53         if a == "wait" return 1.0 end
54         if a == "test" return 0.8 end
55         if a == "treat" return 0.1 end
56     end
57 )
58
59
60
61 const policy = FunctionPolicy(o->"wait")
62 sim = RolloutSimulator(max_steps=100)
63 @show mean(POMDPs.simulate(sim, cancer, policy) for _ in 1:10_000)

```

Problem 3:

I chose to implement DQN for my deep RL algorithm. While for the most part, my DQN was a standard approach to the algorithm, there were a few design decisions I made so that the algorithm achieved an expected reward above 40. One of these decisions was to implement a warmup period before training the neural network. My idea was that this should give the model more time to explore actions freely, and will ensure the network has enough data before it began training. In an attempt to improve performance, I implemented a method that saves the best Q-network seen throughout all the episodes. The code would then train the other networks on this Q network. That change allowed my implementation of DQN to reach a best score of 53.4.

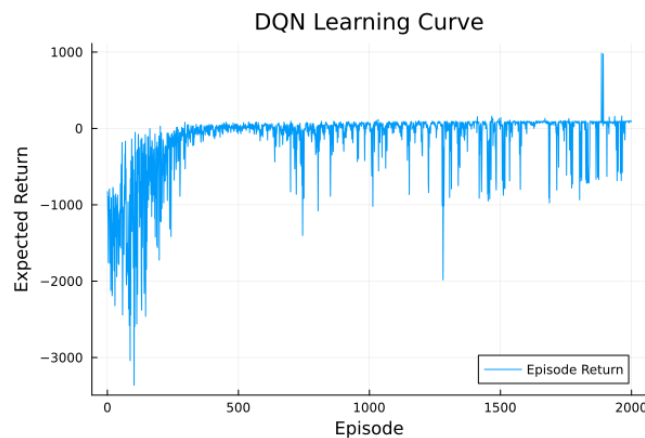


Figure 3: DQN Learning Curve

```

1 using DMUStudent.HW5: HW5, mc
2 using QuickPOMDPs: QuickPOMDP
3 using POMDPTools: Deterministic, Uniform, SparseCat, FunctionPolicy,
  RolloutSimulator
4 using Statistics: mean
5 using CommonRLInterface
6 using Flux
7 using Random
8 using ProgressMeter
9 using CommonRLInterface.Wrappers: QuickWrapper
10 using ElectronDisplay

```

```

11 ElectronDisplay.CONFIG.single_window = true
12 import POMDPs
13
14
15 env = QuickWrapper(HW5.mc,
16                     actions=[-1.0, -0.5, 0.0, 0.5, 1.0],
17                     observe=mc->observe(mc)[1:2])
18
19 function loss(Q, Q_target, s, a_ind, r, sp, done)
20     if ! done
21         return (r + 0.99*maximum(Q_target(sp)) - Q(s)[a_ind])^2
22     else return (r-Q(s)[a_ind])^2 end
23 end
24
25 function policy(env,s,eps,Q)
26     if rand() < eps
27         return rand(1:length(actions(env)))
28     else
29
30         return argmax(i -> Q(s)[i], 1:length(actions(env)))
31     end
32 end
33
34 function simulate(env,Q)
35     reset!(env)
36     r_tot = 0
37     done = false
38     count = 0
39     eps = 0
40     MAX_STEPS = 1000
41
42     while !done && count < MAX_STEPS
43         s = observe(env)
44         a_ind = policy(env,s,eps, Q)
45         r = act!(env, actions(env)[a_ind])
46         sp = observe(env)
47         done = terminated(env)
48         r_tot += (0.99^count) * r
49         s = sp
50         count += 1
51     end
52     return r_tot / count
53 end
54
55 function dqn(env)
56     Q = Chain(Dense(2,128), relu,
57               Dense(128, length(actions(env)))) # Tune as needed
58     opt = Flux.setup(ADAM(0.0005), Q)
59
60     Q_target = deepcopy(Q)
61     N_EPSISODES = 2000
62     MAX_STEPS = 10000
63     WARMUP_STEPS = 2000 # Idea is to let it explore for a while to gain
                          # experience_tuples before training to ensure NN has data

```

```

64  REPLAY_CAPACITY = 200_000
65  TRAIN_SAMPLES_PER_EPISODE = 2_000
66
67  buffer = []
68  learning_curve = Float64[]
69
70
71  @showprogress for i = 1:N_EPSISODES
72
73      reset!(env)
74      count = 0
75      eps = max(0.01, 1-i/N_EPSISODES)
76      episode_return = 0
77      done = false
78
79      while !done && count < MAX_STEPS
80          # Create our experience tuple
81          s = observe(env)
82          a_ind = policy(env,s,eps,Q)
83          r = act!(env, actions(env)[a_ind])
84          sp = observe(env)
85          done = terminated(env)
86          episode_return += r
87          experience_tuple = (s, a_ind, r, sp, done)
88          push!(buffer, experience_tuple)
89          if length(buffer) > REPLAY_CAPACITY
90              popfirst!(buffer) # drop oldest transition
91          end
92
93          count += 1
94      end
95
96      ## Save deepcopy only if Q policy is better than prev Q
97      r_now = simulate(env,Q)
98
99      push!(learning_curve, episode_return)
100     r_trial = simulate(env,Q_target)
101
102     if r_now > r_trial # This will ensure Q_target is always a better
103         Q than the previous one
104
105         Q_target = deepcopy(Q)
106     end
107
108     Q_target = deepcopy(Q)
109
110     if length(buffer) >= WARMUP_STEPS
111         n_updates = min(TRAIN_SAMPLES_PER_EPISODE, length(buffer))
112         for data in rand(buffer, n_updates)
113             # this runs a forward and backward pass to calculate the
114             loss and gradient
115             loss_value, grads = Flux.withgradient(loss, Q, Q_target,
116                 data...)

```

```

115
116         # this will take a gradient step
117         Flux.update!(opt, Q, grads[1])
118     end
119 end
120
121     ## Stuff for learning curve
122 end
123 return Q, learning_curve
124 end
125
126 Q, learning_curve = dqn(env)
127
128 scr = HW5.evaluate(s->actions(env)[argmax(Q(s[1:2]))], n_episodes=10_000).
    score
129 println("\n_score_=$scr")
130 #if scr > 40
131 #     HW5.evaluate(s->actions(env)[argmax(Q(s[1:2]))], "bijan.
    jourabchi@colorado.edu") # you will need to remove the n_episodes=100
    keyword argument and add your email as a positional argument to create
    a json file; evaluate needs to run 10_000 episodes to produce a json
132 #end
133 #-----
134 # Rendering
135 #-----
136
137 # You can show an image of the environment like this (use ElectronDisplay
    if running from REPL):
138 ElectronDisplay.display(render(env))
139
140 # The following code allows you to render the value function
141 using Plots
142 lc_plot = plot(
143     1:length(learning_curve),
144     learning_curve,
145     xlabel="Episode",
146     ylabel="Expected_Return",
147     title="DQN_Learning_Curve",
148     label="Episode_Return"
149 )
150 savefig(lc_plot, "DQN_LC.png")
151
152 xs = -3.0f0:0.1f0:3.0f0
153 vs = -0.3f0:0.01f0:0.3f0
154 heatmap(xs, vs, (x, v) -> maximum(Q([x, v])), xlabel="Position(x)",
    ylabel="Velocity(v)", title="Max_Q_Value")

```